

```

1  module ALU_Controller (
2      input wire      clk,
3      input wire      reset,
4      input wire      en_fir,
5      input wire [7:0] rj_L_data,
6      input wire [7:0] rj_R_data,
7      input wire [15:0] coeff_L_data,
8      input wire [15:0] coeff_R_data,
9      input wire      done_shifting,
10
11     output reg        clear,
12     output reg        clear_reg,
13     output reg        load_adder,
14     output reg        load_shift,
15     output reg        shift,
16
17     output reg [7:0]   read_addr_data_L,
18     output reg [7:0]   read_addr_data_R,
19     output reg [3:0]   read_addr_rj_L,
20     output reg [3:0]   read_addr_rj_R,
21     output reg [4:0]   read_addr_coeff_L,
22     output reg [4:0]   read_addr_coeff_R
23 );
24
25 // FSM state definitions
26 reg [2:0] state, next_state;
27 localparam IDLE          = 3'd0,
28              LOAD_RJ      = 3'd1,
29              LOAD_COEFF   = 3'd2,
30              CALC_INDEX   = 3'd3,
31              SHIFT_DATA   = 3'd4,
32              WAIT_SHIFT_DONE = 3'd5,
33              INCREMENT    = 3'd6,
34              DONE         = 3'd7;
35
36 // Internal registers
37 reg [9:0] n_L, n_R;
38 reg [3:0] tap_L, tap_R;
39 reg [4:0] coeff_L_count, coeff_R_count;
40 reg [7:0] rj_L_value, rj_R_value;
41 reg [7:0] coeff_delay_L, coeff_delay_R;
42 reg signed [8:0] index_L, index_R;
43
44 // Sequential logic: FSM state and counters
45 always @(posedge clk or posedge reset) begin
46     if (reset) begin
47         state <= IDLE;
48         n_L <= 0; n_R <= 0;
49         tap_L <= 0; tap_R <= 0;
50         coeff_L_count <= 0; coeff_R_count <= 0;
51         rj_L_value <= 0; rj_R_value <= 0;
52         coeff_delay_L <= 0; coeff_delay_R <= 0;
53         index_L <= 0; index_R <= 0;
54     end else begin
55         state <= next_state;
56
57         case (state)
58             LOAD_RJ: begin
59                 rj_L_value <= rj_L_data;
60                 rj_R_value <= rj_R_data;
61             end
62
63             LOAD_COEFF: begin
64                 coeff_delay_L <= coeff_L_data[7:0];
65                 coeff_delay_R <= coeff_R_data[7:0];
66             end
67
68             CALC_INDEX: begin
69                 index_L <= n_L - coeff_delay_L;

```

```

70         index_R <= n_R - coeff_delay_R;
71     end
72
73     INCREMENT: begin
74         // LEFT loop logic
75         if (coeff_L_count < rj_L_value - 1)
76             coeff_L_count <= coeff_L_count + 1;
77         else begin
78             coeff_L_count <= 0;
79             if (tap_L < 15)
80                 tap_L <= tap_L + 1;
81             else begin
82                 tap_L <= 0;
83                 if (n_L < 999)
84                     n_L <= n_L + 1;
85             end
86         end
87
88         // RIGHT loop logic
89         if (coeff_R_count < rj_R_value - 1)
90             coeff_R_count <= coeff_R_count + 1;
91         else begin
92             coeff_R_count <= 0;
93             if (tap_R < 15)
94                 tap_R <= tap_R + 1;
95             else begin
96                 tap_R <= 0;
97                 if (n_R < 999)
98                     n_R <= n_R + 1;
99             end
100         end
101     end
102 endcase
103 end
104 end
105
106 // Combinational FSM logic
107 always @(*) begin
108     // Default outputs
109     clear = 0;
110     clear_reg = 0;
111     load_adder = 0;
112     load_shift = 0;
113     shift = 0;
114
115     read_addr_rj_L = tap_L;
116     read_addr_rj_R = tap_R;
117     read_addr_coeff_L = coeff_L_count;
118     read_addr_coeff_R = coeff_R_count;
119     read_addr_data_L = 8'd0;
120     read_addr_data_R = 8'd0;
121
122     next_state = state;
123
124     case (state)
125     IDLE: begin
126         if (en_fir) begin
127             clear = 1;
128             clear_reg = 1;
129             next_state = LOAD_RJ;
130         end
131     end
132
133     LOAD_RJ: begin
134         next_state = LOAD_COEFF;
135     end
136
137     LOAD_COEFF: begin
138         next_state = CALC_INDEX;

```

```

139         end
140
141         CALC_INDEX: begin
142             load_adder = 1;
143             read_addr_data_L = index_L[7:0];
144             read_addr_data_R = index_R[7:0];
145             next_state = SHIFT_DATA;
146         end
147
148         SHIFT_DATA: begin
149             load_shift = 1;
150             next_state = WAIT_SHIFT_DONE;
151         end
152
153         WAIT_SHIFT_DONE: begin
154             if (done_shifting)
155                 next_state = INCREMENT;
156         end
157
158         INCREMENT: begin
159             if (n_L >= 999 && n_R >= 999)
160                 next_state = DONE;
161             else
162                 next_state = LOAD_RJ;
163         end
164
165         DONE: begin
166             next_state = IDLE;
167         end
168     endcase
169 end
170
171 endmodule
172
173
174 //ALU_Controller TestBench
175
176 `timescale 1ns / 1ps
177
178 module ALU_Controller_tb;
179
180     reg clk, reset, en_fir;
181     reg [7:0] rj_L_data, rj_R_data;
182     reg [15:0] coeff_L_data, coeff_R_data;
183     reg done_shifting;
184
185     wire clear, clear_reg, load_adder, load_shift, shift;
186     wire [7:0] read_addr_data_L, read_addr_data_R;
187     wire [3:0] read_addr_rj_L, read_addr_rj_R;
188     wire [4:0] read_addr_coeff_L, read_addr_coeff_R;
189
190     // Instantiate DUT
191     ALU_Controller uut (
192         .clk(clk),
193         .reset(reset),
194         .en_fir(en_fir),
195         .rj_L_data(rj_L_data),
196         .rj_R_data(rj_R_data),
197         .coeff_L_data(coeff_L_data),
198         .coeff_R_data(coeff_R_data),
199         .done_shifting(done_shifting),
200         .clear(clear),
201         .clear_reg(clear_reg),
202         .load_adder(load_adder),
203         .load_shift(load_shift),
204         .shift(shift),
205         .read_addr_data_L(read_addr_data_L),
206         .read_addr_data_R(read_addr_data_R),
207         .read_addr_rj_L(read_addr_rj_L),

```

```

208     .read_addr_rj_R(read_addr_rj_R),
209     .read_addr_coeff_L(read_addr_coeff_L),
210     .read_addr_coeff_R(read_addr_coeff_R)
211 );
212
213 // Clock generation
214 always #5 clk = ~clk;
215
216 initial begin
217     // Initial state
218     clk = 0;
219     reset = 1;
220     en_fir = 0;
221     rj_L_data = 8'd2;           // 2 taps
222     rj_R_data = 8'd2;
223     coeff_L_data = 16'h0001; // delay = 1
224     coeff_R_data = 16'h0001;
225     done_shifting = 0;
226
227     // Reset pulse
228     #10 reset = 0;
229
230     // Start FIR operation
231     #10 en_fir = 1;
232     #10 en_fir = 0;
233
234     // Simulate shifter done after CALC_INDEX → SHIFT → WAIT
235     repeat (3) begin
236         wait (load_shift);
237         #10 done_shifting = 1;
238         #10 done_shifting = 0;
239     end
240
241     // Allow more cycles to show control progression
242     #100;
243     $finish;
244 end
245
246 // Optional FSM output monitoring
247 initial begin
248     $monitor("T=%0t | clr=%b load_add=%b load_shf=%b shift=%b RJ_L=%d COEFF_L=%d
249             DATA_L_ADDR=%d",
250             $time, clear, load_adder, load_shift, shift,
251             read_addr_rj_L, read_addr_coeff_L, read_addr_data_L);
252 end
253 endmodule
254

```